Problem 1:

Consider Problem 1 described in Project 2. The goal is to analyze the performance of the temporal difference learning algorithms. Set $p = 0.025$, $\gamma = 0.96$, $\alpha = 0.25$, $\epsilon = 0.1$, the total number of episodes 1,000, the maximum number of steps if the agent does not reach the goal state 1,000 (i.e., the maximum length of each episode). Any parameter that is not specified can be set/tuned by you, and you need to include them in your final report.

Implement Q-Learning, SARSA and Tabular actor-critic algorithm ($\beta = 0.05$) algorithms. For each algorithm, you need to run the algorithm for 10 independent runs. Report your results in terms of:

1. How many times among 10 independent runs, upon the termination of learning, a path from start to goal has been obtained?
2. Show the optimal policy and the optimal path from start to goal for one of the 10 independent runs (use the results of a run that the path from start to goal is found; if the path is not found in all runs, show the results from a random run).
3. Show the average accumulated reward (in 10 independent runs) with respect to the episode number. See Figure 3.
4. Plot the average accumulated reward with respect to the episode number for all algorithms in a single plot. Describe your findings.
5. For each algorithm, track during training at which episode the greedy policy (derived from the current Q-values or actor) first produces a valid path from start to goal. Report this episode number for each algorithm. If the path is never found during training, record it as 1,000. Compare convergence speed across algorithms.
6. Choose one algorithm and vary the learning rate $\alpha \in \{0.05, 0.1, 0.25, 0.5\}$ while keeping all other parameters fixed. For each value of α, run 10 independent runs and plot the average accumulated reward curves on a single figure. Briefly discuss how α affects convergence speed and stability.

*For any of the algorithms that in all runs the path from start to goal is not found, you need to change the parameters (e.g., α, β, ϵ, episode number, length of the episode) in a trial and error and report the best solutions found.

SARSA Algorithm

Algorithm 1: SARSA

```
Initialize Q(s, a) arbitrarily;
repeat
    Initialize s;
    Choose a from s using policy derived from Q (e.g., ϵ-greedy);
    repeat                                                  // for each step of episode
        Take action a, observe r, s';
        Choose a' from s' using policy derived from Q (e.g., ϵ-greedy);
        Q(s, a) ← Q(s, a) + α[r + γQ(s', a') − Q(s, a)];
        s ← s';
        a ← a';
    until s is terminal;
until;
```

Q-Learning Algorithm

Algorithm 2: Q-Learning

```
Initialize Q(s, a) arbitrarily;
repeat
    Initialize s;
    repeat                                                  // for each step of episode
        Choose a from s using policy derived from Q (e.g., ϵ-greedy);
        Take action a, observe r, s';
        Q(s, a) ← Q(s, a) + α[r + γ max_a' Q(s', a') − Q(s, a)];
        s ← s';
    until s is terminal;
until;
```

Tabular Actor-Critic Algorithm

Algorithm 3: Tabular Actor-Critic

$V(s) = 0$, $H(s,a) = 0$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}$;
repeat `// for` N `episodes`
  Start from a random state $s_0 \in \mathcal{S}$, $t = 0$;
  while $t < T$ (episode length) **do**
    Select action: $a_t \sim \pi(\cdot \mid s_t)$ where $\pi(a \mid s) = \dfrac{e^{H(s,a)}}{\sum_{a' \in \mathcal{A}} e^{H(s,a')}}$;
    Take action a_t, move to state s_{t+1} and observe R_{t+1};
    $\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$;
    $V(s_t) = V(s_t) + \alpha \delta_t$;
    $H(s_t, a_t) = H(s_t, a_t) + \beta \delta_t \big(1 - \pi(a_t \mid s_t)\big)$;
    $t = t + 1$;
  end
until;

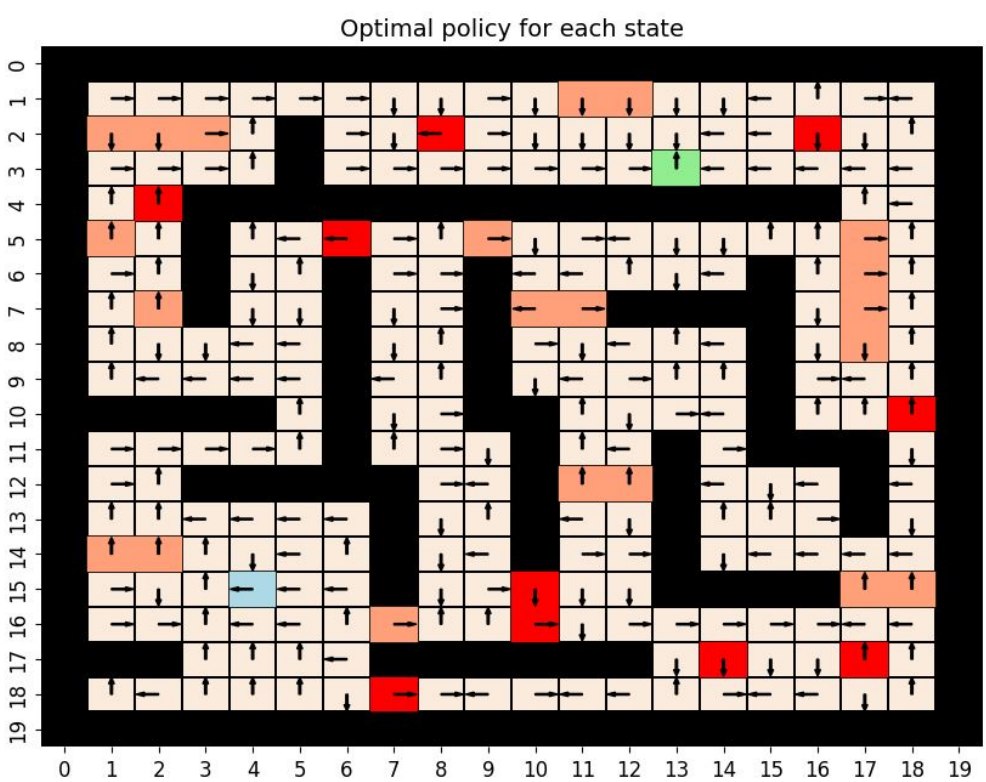

Figure 1: The optimal policy obtained by SARSA for $\alpha = 0.3$ and $\epsilon = 0.1$.

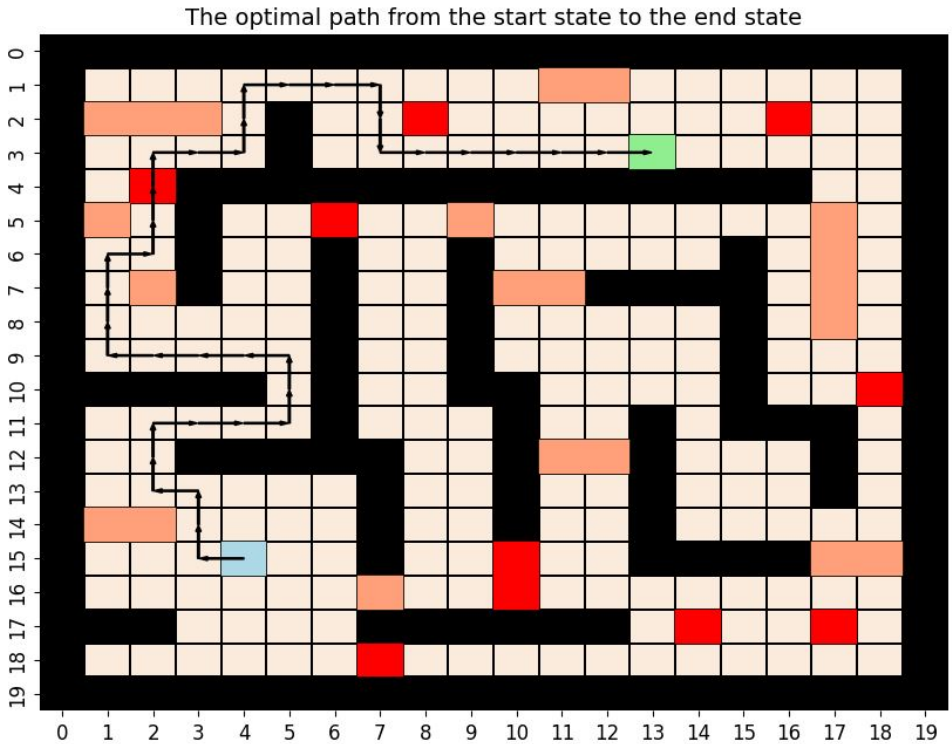

Figure 2: The optimal path obtained by SARSA for $\alpha = 0.3$ and $\epsilon = 0.1$.

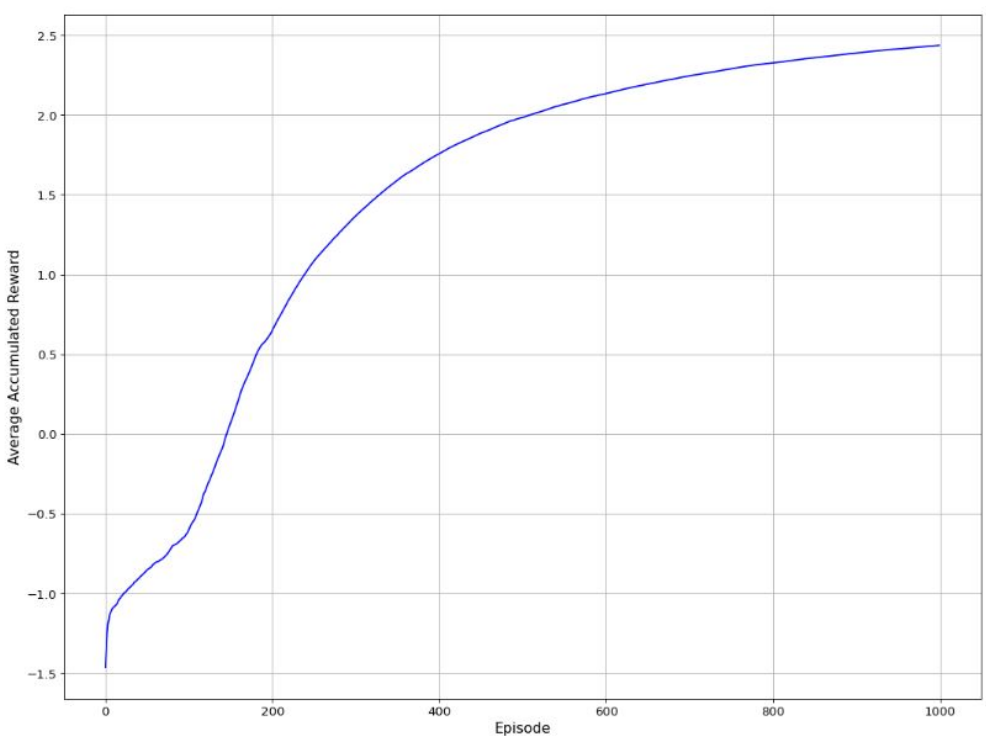

Figure 3: The average accumulated reward with respect to the episode obtained by SARSA for $\alpha = 0.3$ and $\epsilon = 0.1$.

Problem 2

Consider Problem 2 described in Project 2. The state space consists of $2^4 = 16$ states and the action space is $A = \{a_1 = [0,0,0,0]^T,\ a_2 = [0,1,0,0]^T,\ a_3 = [0,0,1,0]^T,\ a_4 = [0,0,0,1]^T\}$. The dynamics of this system is governed by the following equation:

$$\boldsymbol{s}_k = \overline{\underbrace{\begin{bmatrix} 0 & 0 & -1 & 0 \\ +1 & 0 & -1 & -1 \\ 0 & +1 & 0 & 0 \\ -1 & +1 & +1 & 0 \end{bmatrix}}_{\textbf{C: Connectivity Matrix}} \boldsymbol{s}_{k-1} \oplus \boldsymbol{a}_{k-1} \oplus \boldsymbol{n}_k} \tag{1}$$

where $\overline{(\cdot)}$ maps the element of vector greater than 0 to 1, and smaller or equal to zero to 0, $\boldsymbol{a}_{k-1}$ is a control input and $\boldsymbol{n}_k$ is the state transition noise.

We assume that four elements of $\boldsymbol{n}_k$ are identically distributed through a Bernoulli noise with parameter p. This can be expressed as:

$$\boldsymbol{n}_k^{(j)} \sim \begin{cases} 1 & \text{with prob } p \\ 0 & \text{with prob } 1-p \end{cases}, \quad \text{for } j = 1, \ldots, 4.$$

The reward function is as follows:

$$R(s, a, s') = 5\, s_1' + 5\, s_2' + 5\, s_3' + 5\, s_4' - c(a)$$

where $c(a)$ denotes the cost of the action a. In this problem, $c(a_1) = c(a_4) = 0$ and $c(a_2) = c(a_3) = 1$. This means that actions a_2 and a_3 have reward of -1.

For creating state-action-reward transitions needed in temporal difference learning algorithms, you need to use Eq. (1). Let's consider the current state is $\boldsymbol{s}_k = [0,1,1,1]^T$, if the action $\boldsymbol{a}_k = [0,0,0,1]^T$ is selected, you need to create $\boldsymbol{n}_k$ by drawing 4 independent samples from Bernoulli(p). For simplicity, let's assume $\boldsymbol{n}_k = [0,0,0,0]^T$. Then the next state can be computed according to Eq. (1) as:

$$\boldsymbol{s}_{k+1} = \overline{\begin{bmatrix} 0 & 0 & -1 & 0 \\ +1 & 0 & -1 & -1 \\ 0 & +1 & 0 & 0 \\ -1 & +1 & +1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}} \oplus \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \overline{\begin{bmatrix} -1 \\ -2 \\ +1 \\ +2 \end{bmatrix}} \oplus \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Set $p = 0.1$, $\gamma = 0.9$, $\alpha = 0.25$, $\epsilon = 0.15$, the total number of episodes 1,000, the maximum number of steps (i.e., the maximum length of each episode) 100.

Implement Q-Learning, SARSA, SARSA$-\lambda$ ($\lambda = 0.95$) and tabular actor-critic ($\beta = 0.05$) algorithms. For each algorithm, you need to run the algorithm for 10 independent runs.

1. Show the optimal policy for all 10 independent runs (10 vectors of size 16, consisting of a_1, a_2, or a_3).
2. Show the average accumulated reward (in 10 independent runs) with respect to the episode number.
3. Plot the average accumulated reward with respect to the episode number for all algorithms in a single plot. Describe your findings.

4. After training, execute the learned greedy policy for each algorithm over 100 episodes. Record how many times each of the 16 states is visited during execution. Present the state visitation counts as a 16×1 vector (or bar chart) for each algorithm. Compare the visitation patterns across algorithms and briefly discuss what they reveal about the quality of the learned policies.

5. For SARSA$-\lambda$, repeat the experiment with $\lambda \in \{0,\ 0.5,\ 0.95\}$ (note that $\lambda = 0$ reduces to standard SARSA). Plot the average accumulated reward curves for the three values of λ on a single figure and briefly discuss the effect of λ on learning performance.

* For any of the algorithms that in all runs the path from start to goal is not found, you need to change the parameters (e.g., $\alpha, \beta, \lambda, \epsilon$, episode number, length of the episode) in a trial and error and report the best solutions found.

SARSA(λ) Algorithm

Algorithm 4: SARSA(λ)

```
Initialize Q(s,a) arbitrarily and e(s,a) = 0, for all s,a;
repeat
    Initialize s,a;
    e(s,a) = 0, for all s,a;
    repeat                                          // for each step of episode
        Take action a, observe r, s';
        Choose a' from s' using policy derived from Q (e.g., ε-greedy);
        δ ← r + γQ(s',a') − Q(s,a);
        e(s,a) ← e(s,a) + 1;
        for all s,a do
            Q(s,a) ← Q(s,a) + α δ e(s,a);
            e(s,a) ← γ λ e(s,a);
        end
        s ← s';;
        a ← a';
    until s is terminal;
until;
```

Questions about the project should be directed to TA, Amirhossein Ravari, at ravari.a@northeastern.edu.